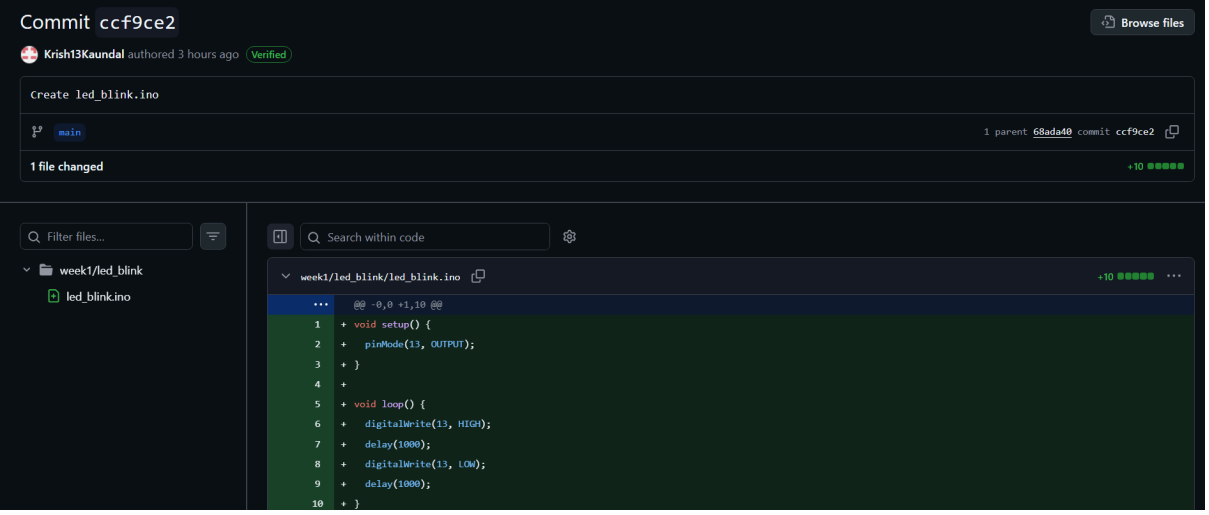


## Module 1

A1 : <https://github.com/Krish13Kaundal/iot-summer-school-2026>

A2 :



Commit ccf9ce2

Krish13Kaundal authored 3 hours ago (Verified)

Create led\_blink.ino

main 1 parent 68ada40 commit ccf9ce2

1 file changed +10

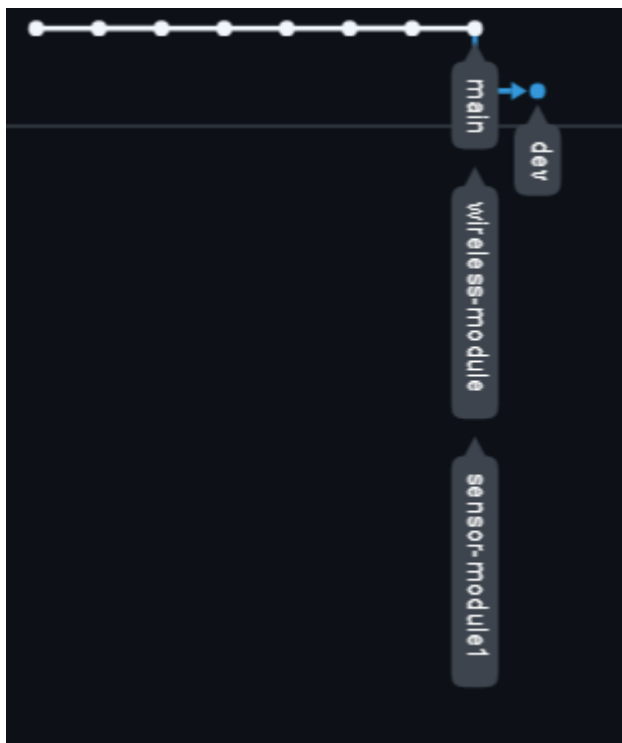
Filter files... Search within code

week1/led\_blink

led\_blink.ino

```
... @@ -0,0 +1,10 @@
1 + void setup() {
2 +   pinMode(13, OUTPUT);
3 + }
4 +
5 + void loop() {
6 +   digitalWrite(13, HIGH);
7 +   delay(1000);
8 +   digitalWrite(13, LOW);
9 +   delay(1000);
10 + }
```

A3:



<https://github.com/Krish13Kaundal/iot-summer-school-2026/commit/8431049f5adbe44e15b5881defe3ae480b57c8c1>

A4:

|                                                                                                                                                                                        |          |         |                                                                                     |                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|---------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>docs: add author header and documentation (v1.3)</b><br> Krish13Kaundal authored 2 hours ago       | Verified | 66d1963 |  |  |
| <b>feat: add potentiometer control for blink speed (v1.2)</b><br> Krish13Kaundal authored 2 hours ago | Verified | 3f9b44b |  |  |
| <b>feat: add serial print for blink count (v1.1)</b><br> Krish13Kaundal authored 2 hours ago          | Verified | 75e0fea |  |  |
| <b>feat: change blink interval</b><br> Krish13Kaundal authored 2 hours ago                            | Verified | 0affebb |  |  |

A5:

 Krish13Kaundal / **iot-summer-school-2026** 

 |  

[Code](#) [Issues](#) **2** [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#) [Settings](#)

  **main**  **iot-summer-school-2026 / week1 / led\_blink / README.md**   

 **Krish13Kaundal** Update README.md fae2b66 · now 

[Preview](#) [Code](#) | [Blame](#) 27 lines (22 loc) · 1.32 KB [Raw](#)     

## IoT LED Blink Project

### Hardware Required

- Arduino Uno / ESP32 Board
- 1x LED (Built-in Pin 13 or external)
- 1x 220-ohm Resistor (if using external LED)
- 1x 10k-ohm Potentiometer
- Breadboard and Jumper Wires

### Circuit Diagram Description

- **LED:** Connected to Digital Pin 13 through a 220-ohm current-limiting resistor to Ground.
- **Potentiometer:** Pin 1 (Left) connected to 5V, Pin 2 (Middle/Wiper) connected to Analog Input Pin A0, and Pin 3 (Right) connected to Ground.

### How to Upload Code (Step-by-Step)

1. Open the Arduino IDE.
2. Open the file `led_blink.ino`.
3. Connect your Arduino board to the computer via USB.
4. Select the correct Board and Port from the **Tools** menu.
5. Click on the **Upload** button (Right arrow icon).

### Expected Output

The LED on Pin 13 will blink. Rotating the potentiometer knob will modify the resistance, changing the delay and altering the blinking speed in real-time. The serial monitor will output the current blink count at a 9600 baud rate.

### Troubleshooting Tips

1. **Board not detected:** Check the USB cable connection and verify if the correct COM port is selected in Arduino IDE.
2. **Serial Monitor showing garbage values:** Ensure that the Serial Monitor baud rate matches `9600`.
3. **LED not blinking:** Double-check the polarity of the external LED (longer leg goes to Pin 13).

A6:

The screenshot shows the GitHub Issues page for the repository 'Krish13Kaundal'. The search bar at the top contains 'is:issue state:open'. Below the search bar, there are filters for 'Open' (2) and 'Closed' (1). The issues list shows two open issues:

- Enhancement: Log data to Serial Monitor in CSV format (Label: enhancement, Assignee: Yourself, Milestone: Week 1 Completion).** #4 · Krish13Kaundal opened 2h ago
- Feature: Add RGB LED support (Label: enhancement, Assignee: Yourself, Milestone: Week 1 Completion).** #3 · Krish13Kaundal opened 2h ago

At the bottom right, there is a 'Give feedback' link.

A8:

The screenshot shows the GitHub Actions workflow page for the repository 'Krish13Kaundal / iot-summer-school-2026'. The workflow is named 'feat: add GitHub Actions workflow for file counting #1'. The status is 'succeeded 2 hours ago in 5s'. The workflow steps are:

- Set up job (1s)
- Checkout Code (1s)
- Run Python script to count .ino files (0s)
- Post Checkout Code (0s)
- Complete job (0s)

The left sidebar shows the workflow file 'count-files' and the 'Run details' section. The 'Annotations' section shows 1 warning.

A9: **Git clone**: Creates a copy of a remote repository on your computer. Use it when working on a project for the first time.

- **Git fetch**: Downloads the latest changes from the remote repository without changing your local files. Use it to review updates before merging.
- **Git pull**: Downloads and automatically merges the latest changes into your local branch. Use it when you want to update your project quickly.

A10: **.gitignore** file tells Git which files and folders should **not** be tracked or uploaded to the repository. It is used to ignore temporary, compiled, and system-generated files.

# Compiled files

\*.hex

\*.elf

# Build folder

build/

# IDE settings

.vscode/

# OS files

.DS\_Store

Thumbs.db

This keeps the repository clean by excluding unnecessary files.

## Module 2

A12:

| Parameters  | Microcontroller | Microprocessor |
|-------------|-----------------|----------------|
| CPU Speed   | 16 Mhz          | 1.5 Ghz        |
| On-chip RAM | 2 KB            | None           |

|                   |          |                                 |
|-------------------|----------|---------------------------------|
| On-chip Flash     | 32 KB    | None                            |
| Operating System  | None     | Linux, Raspberry Pi OS, Windows |
| Typical Use Case  | Simple   | Complex                         |
| Power Consumption | Very Low | High                            |

## A13: **Arduino UNO Pins and Their Uses**

### 1. Digital Input

Digital input pins read only **HIGH (1)** or **LOW (0)** signals. They are used to receive input from devices such as push buttons or switches.

**Example:** Push button.

**IoT Use Case:** Detecting when a smart door is opened or closed.

### 2. Digital Output

Digital output pins send **HIGH** or **LOW** signals to control external devices.

**Example:** LED.

**IoT Use Case:** Turning a relay ON/OFF to control home appliances.

### 3. Analog Input

Analog input pins read voltages between **0V and 5V** and convert them into digital values.

**Example:** Temperature sensor or potentiometer.

**IoT Use Case:** Monitoring temperature in a smart greenhouse.

### 4. PWM Output

PWM (Pulse Width Modulation) pins provide variable output to control brightness or motor speed.

**Example:** LED brightness control.

**IoT Use Case:** Controlling the speed of a smart cooling fan.

### 5. I2C/SPI Pins

These pins are used for communication between the Arduino and sensors or modules. **I2C** uses **A4 (SDA)** and **A5 (SCL)**, while **SPI** uses **pins 10–13**.

**Example:** OLED display or RFID module.

**IoT Use Case:** Connecting multiple sensors in a smart weather monitoring system.

A14: <https://www.tinkercad.com/things/fABb35qI1M0-traffic-signal>

A15: <https://www.tinkercad.com/things/1GtdTmeDpCL-piano>

A16: <https://www.tinkercad.com/things/crwcuGQBS87-serial-command>

A17: <https://www.tinkercad.com/things/cH4hZhVvfxE-pwm-fading-modes>

A18: <https://www.tinkercad.com/things/2xjj5uDY3H4-vending-machine>

A19: **analogRead()** is used to read analog values from sensors through the analog input pins. It returns a value between **0 and 1023**.

**IoT Example:** Reading data from a temperature sensor in a smart weather station.

**analogWrite()** is used to generate a **PWM (Pulse Width Modulation)** signal on PWM pins. It accepts values from **0 to 255** to control output levels.

**IoT Example:** Adjusting the brightness of a smart street light or controlling the speed of a cooling fan.

**PWM (Pulse Width Modulation)** is a technique that simulates analog output by rapidly switching a digital signal ON and OFF. It is mainly used to control LED brightness, motor speed, and similar devices.

A20: **setup()** runs **once** when the Arduino starts. It is used to initialize pins, sensors, and communication.

**loop()** runs **continuously** after **setup()**. It contains the main program that keeps repeating.

If a **long delay()** is used inside **loop()**, the Arduino stops executing other tasks during that time. As a result, sensor readings become slow and the system responds late to changes.

The **non-blocking alternative** is to use the **millis()** function. It allows the Arduino to perform multiple tasks at the same time without stopping the program, making sensor readings more responsive.

## MODULE 3

A21: <https://wokwi.com/projects/468724597492714497> (DHT22 is used)

A22: <https://wokwi.com/projects/468706745095478273>

A23: <https://wokwi.com/projects/468717761668855809>

A24: <https://wokwi.com/projects/468725158561673217> (DHT22 is used)

A25:<https://www.tinkercad.com/things/fwZPyjN7BFU-servo-motor>

A26:<https://www.tinkercad.com/things/9tapsOvL1d-dc-motor> (I293D sensor is used)

A27:<https://wokwi.com/projects/468725731864399873> (DHT22 is used)

A28:<https://wokwi.com/projects/468723882120539137>

A29:**Sensor calibration** is the process of adjusting a sensor so that it provides accurate and reliable readings. It is important in IoT systems because incorrect sensor values can lead to wrong decisions and poor system performance.

For an analog soil moisture sensor, the two-point calibration method is used:

- Dry soil: Record the sensor value in completely dry soil (0% moisture).
- Wet soil: Record the sensor value in fully wet soil (100% moisture).

These two values are then used to convert future sensor readings into accurate moisture percentages.

A30:**I2C (Inter-Integrated Circuit)** is a communication protocol that allows multiple sensors and devices to communicate with a microcontroller using only two wires.

- SDA (Serial Data): Transfers data between devices.
- SCL (Serial Clock): Provides the clock signal to synchronize communication.

Each I2C device has a unique address. The Arduino communicates with a specific device by sending its address before exchanging data.

#### **Common I2C IoT sensors:**

1. BME280 (Temperature, Humidity, and Pressure Sensor)
2. MPU6050 (Accelerometer and Gyroscope)
3. BH1750 (Light Intensity Sensor)